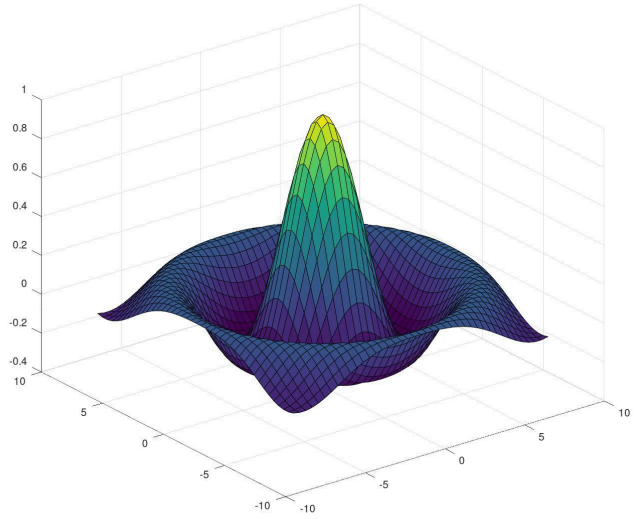
OCTAVE/MATLAB



Comandos básicos para as disciplinas de Modelagem e Simulação e Controle

Prof. Lucas Molina
lmolina@academico.ufs.br

Matlab ou Octave?

Todos que me conhecem sabem que eu sou fã do Matlab. Se eu pudesse, usaria o Matlab como nosso ambiente de simulação nas nossas disciplinas. No entanto, o custo de aquisição do Matlab institucionalmente é proibitivo e me obrigou a utilizar nas aulas um software gratuito. Entre as opções disponíveis escolhi o Octave por ser um "clone" do Matlab, o que significa que quaisquer códigos antigos que vocês tenham em Matlab podem ser utilizados no Octave com quase nenhuma ou nenhuma alteração. Além disso, aqueles que preferirem utilizar o Matlab ao invés do Octave poderão acompanhar as aulas normalmente.

É importante destacar que nas disciplinas listadas aqui e ministradas por mim, o uso desses ambientes tem o propósito de facilitar a manipulação de dados matriciais, mas muitas das ferramentas que aprenderemos a projetar e implementar computacionalmente na nossa disciplina existem implementadas nesses ambientes. Nesse contexto, a utilização de funções que não são listadas aqui deve passar pelo meu aval e deve ainda ser uma função que exista no ambiente do Octave, para que eu possa acompanhar os resultados de vocês, caso vocês optem em usar o Matlab.

A partir daqui, tratarei como se estivéssemos todos utilizando o Octave.

Instalando o Octave 1

Para instalar, basta baixar o Octave pelo site:
https://www.gnu.org/software/octave/index

A instalação irá adicionar dois links na área de trabalho: um para rodar o Octave em linha de código, GNU Octave (CLI), e outra para rodar o Octave com uma interface gráfica similar ao Matlab, GNU Octave (GUI). É nessa última que eu pretendo trabalhar.

Janelas do Octave e configurações iniciais:

As janelas do Octave assim como algumas configurações iniciais que eu fiz para poder aumentar a similaridade e a compatibilidade com os códigos do Matlab eu apresento pra vocês na videoaula sobre configurações iniciais do Octave.

Janela de Comandos e variáveis:

- Nessa janela, o símbolo do prompt de comando (>>) indica que o programa está pronto aguardando novas instruções.
- As setas para cima e para baixo permitem acessar o histórico recente de comandos.
- Funciona como uma calculadora, você digita a expressão que quer calcular e digita enter e o resultado é retornado na mesma janela e/ou armazenado em uma variável criada por você.
- O Octave é case sensitive, ou seja, letras maiúsculas e minúsculas representam variáveis ou funções diferentes.
- **clc** - limpa a janela de comando.
- **%** - comentário. Toda linha iniciado com % na janela de comando ou em um script não será executada pelo Octave.
- **;** - inibe o retorno de uma operação. Qualquer operação que termine com **;** o valor da operação será armazenado na variável determinada mas o resultado não será retornado na janela de comando.
- = - operador de atribuição. As variáveis não precisam ser declaradas. A depender do valor que foi atribuído para uma variável o Octave automaticamente define o tipo e tamanho da variável.
- O nome das variáveis deve começar com letras, mas pode conter números o nome. Tome cuidado para não sobrescrever uma variável predefinida do Octave ou mesmo uma função dele.
- **clear** - apaga as variáveis da memória do Octave.

Comandos Básicos

Operadores Aritméticos

- (+ , - , * , / , ^) são os operadores de adição, subtração, multiplicação, divisão e potência, respectivamente.
- A regra de procedência vale primeiramente o que está entre parênteses, depois a potência, divisão, multiplicação, adição e subtração, nessa ordem.

Algumas variáveis predefinidas:

- **ans** - variável à qual é atribuído o resultado de uma operação que não foi atribuído a uma variável.
- **pi** - π.
- **eps** - menor diferença entre dois números que o Octave consegue mensurar.
- **inf** - infinito.
- **NaN** = Not a Number. Retornada em uma operação quando o resultado dela deu algo que não é um número (deu muito errado!!!)
- **i, j, 1j e 1i** - diferentes formas de representar a parte imaginária de um número complexo.

Algumas funções elementares:

As funções do Octave aqui listadas operam sobre diferentes tipos de argumentos. Pesquisem na ajuda do Octave os tipos de argumentos e como usar essas funções e faça alguns testes com elas.

- **sqrt** - raiz quadrada.
- **exp** - exponencial.
- **abs** - valor absoluto.
- **log** - logaritmo neperiano.
- **log10** - logaritmo na base 10.
- **factorial** - fatorial.
- **sin, cos, tan** - seno, cosseno e tangente, respectivamente.
  - Funções trigonométricas sempre consideram ângulos em radiano.
- **asin, acos, atan** - arco seno, arco cosseno e arco tangente, respectivamente.
- **atan2** - função arco tangente definida em todo o círculo trigonométrico.
- **tanh** - função tangente hiperbólica
- **round** - arredondamento para o inteiro mais próximo.
- **sign** - função sinal.
- **rand** - gera um número “aleatório” com distribuição uniforme.
- **randn** - gera um número “aleatório” com distribuição normal.
- **conj** - calcula o conjugado de um número complexo.
- **real** - valor da parte real de um número complexo.

- **imag** - valor da parte imaginária de um número complexo.
- **angle** - calcula o ângulo de um número complexo.

Arranjos (vetores e matrizes):

- Assim como qualquer variável, não é preciso declarar o tamanho e o tipo dos arranjos.

```Octave
# Criando arranjos com elementos espaçados por um valor constante:

V = [1:0.1:10];

# Criando arranjos com uma lista de números conhecidos:

V = [1 2 3]; # Vetor Linha
V = [1; 2; 3]; # Vetor Coluna
V = [1, 2, 3]; # Vetor Linha

# Vetor Coluna
V = [1
2
3];
```

- **linspace** - cria arranjos com elementos linearmente espaçados.
- **logspace** - cria arranjos com elementos logaritmicamente espaçados.
- **zeros** - cria um arranjo cujos elementos são 0s.
- **ones** - cria um arranjo cujos elementos são 1s.
- **eye** - cria uma matriz identidade.
- **diag** - coloca um vetor na diagonal de uma matriz diagonal e coloca a diagonal de uma matriz em um vetor (depende do argumento).
- **.'** (ponto aspa simples) - retorna o transposto de um arranjo
- **'** (aspa simples) - retorna o conjugado transposto de um arranjo
- **inv** - retorna a inversa de uma matriz
- **det** - retorna o determinante de uma matriz
- **size** - retorna o tamanho de um arranjo
- **length** - retorna o comprimento de um arranjo
- **mean** - calcula a média.
- **var** - calcula a variância.
- **std** - calcula o desvio padrão.
- **min** - retorna o menor valor.
- **max** - retorna o maior valor.
- **sum** - somatório dos elementos.

- **sort** - reordena os elementos.

Referência a elementos de arranjos:

```
>> V =
>>    1   2   3
>>    4   5   6
>> V(1)
ans = 1
>> V(5)
ans = 3
>> V(3)
ans = 2
>> V(2,2)
ans = 5
>> V(2,:) # Acessar a segunda linha inteira
ans =
   4   5   6
>> V(2,2:3) # Acessar da segunda à terceira coluna da linha 2
ans =
   5   6
```

Eliminando e adicionando elementos a um arranjo:

```
>> V = [1 2 3; 4 5 6] #Criando o vetor V
V =

    1   2   3
    4   5   6

>> V(2,3) = 10 # Substituir o elemento da linha 2, coluna 3
V =

    1   2   3
    4   5  10

>> V(3,3) = 7 # Adicionar o elemento da linha 3, coluna 3
V =

    1   2   3
    4   5  10
```

```
    0    0    7

>> V(:,3) = [] # Atribuindo o vazio para a terceira coluna (eliminar)
V =

    1    2
    4    5
    0    0
```

Operações entre arranjos. As operações entre arranjos podem ser de dois tipo: respeitando a álgebra linear, através dos operadores tradicionais de adição, subtração, multiplicação e potência, ou através de operações ponto a ponto, onde os vetores envolvidos devem ter a mesma dimensão:

```Octave
>> A = [1 2; 3 4];
>> B = [2 2; 2 2];
>> A*B
ans =

     6     6
    14    14

>> A.*B
ans =

    2    4
    6    8

>> A.^B
ans =

     1     4
     9    16
```

- **Concatenação**. Para concatenar vetores ou matrizes basta colocá-los como elementos de um vetor. Seja um vetor **V** e um vetor **W**, então a matriz A resultado da concatenação de **V** e **W** pode ser escrita por **A = [V , W]**.

Rotinas m-file (Script) (Editor):

- Funciona como uma sequência de comandos feita na Janela de Comando, basta executar pelo editor.
- É interessante pois permite organizar melhor na forma de um programa e inserir comentários, além de acompanhar a evolução do programa passo a passo no modo DEBUG, uma das grandes funcionalidades desse ambiente. O modo debug será ilustrado em uma videoaula sobre o tema disponibilizada junto com esse material.
- **input** - solicitação de entrada de informação pela janela de comando.
- **disp** - apresentar informação na janela de comando
- **save** - salvar dados em um arquivo **.mat** para posterior utilização
- **load** - carregar dados salvos em um arquivo **.mat**

Arquivos m-file como funções (Editor):

- Arquivos **.m** podem ser sequências de comandos executados diretamente do script como também podem ser funções destinadas a serem utilizadas por outros scripts ou chamadas na janela de comando.
- O arquivo **.m** deve ter o mesmo nome da função que por sua vez deve ser declarada na primeira linha do arquivo (isso já não é necessário em versões mais novas do MATLAB, por exemplo).

Abaixo um exemplo de função ilustrando sua sintaxe com variáveis de entrada e de saída:

```Octave
# O NOME DO ARQUIVO DO CÓDIGO É "produto.m"

function C = produto (A,B)
  C = A*B;
end
```

Abaixo um outro exemplo ilustrando funções aninhadas com declaração no mesmo arquivo (função local):

```Octave
 # O NOME DO ARQUIVO DO CÓDIGO É "produto_estranho.m"

 function C = produto_estranho (A,B)
   B = metade(B);
   C = A*B;
 end

 function D = metade (E)
```

```
9     D = E/2;
10  end
```

As variáveis que existem na função são aquelas atribuídas como parâmetro de entrada da função ou aquelas criadas nela. Todas as variáveis de uma função, a priori, são variáveis locais. Para criar uma variável global, ela deve ser declarada como tal, assim como ilustrado no exemplo abaixo:

```Octave
1   # O NOME DO ARQUIVO DO CÓDIGO É "produto_estranho.m"
2
3   function C = produto_estranho (A,B)
4     global X
5     X = A;
6     B = add_A(B);
7     C = A*B;
8   end
9
10  function D = add_A(E)
11    global X
12    D = E+X;
13  end
```

Elementos de programação:

Operadores lógicos e relacionais:

- < (menor), > (maior), <= (menor ou igual), >= (maior ou igual).
- == (igual a), = (diferente de).
- & (AND), | (OR), (NOT).

Sintaxe da estrutura condicional IF:

```Octave
if (a >= 5 & b == 3)
  c = 4;
  if a > 7
    d = 2;
  else
    d = 1;
  end
end
```

Sintaxe dos laços FOR e WHILE:

```Octave
for k = 1:2:5 #para k de 1 a 5 de 2 em 2
  k
end

k = 1;
while(1) #fazer indefinidamente
  k = k+1
end

k = 1;
while(k <5) # fazer enquanto k menor que 5
  k = k+1
end
```

Outros comandos:

- **break** - saída imediata do loop for ou while mais interno.
- **continue** - parte para a próxima iteração do loop for ou while mais interno.

Gráficos 2D

- Nos plots de gráficos, o Octave recebe como argumento uma sequência de pontos. Cada ponto do plot pode ser marcado por um símbolo (marcador) e a conexão entre os pontos é feita sempre com uma reta que pode ser desenhada com diferentes tipos de linha. Tanto os marcadores quanto as linhas podem ser plotadas de diferentes cores.
- **figure** - esse comando abre uma nova figura para o próximo plot.
- Existem muitos atributos para o plot que podem ser encontrados na ajuda do Octave. Aqui apresentarei alguns atributos simplificados que permitem selecionar de forma rápida cores, marcadores e tipos de linha.
- Por padrão, o plot é feito com uma linha contínua, sem marcadores nos pontos e na cor azul.

	Cor da linha e do marcador
G	Verde
R	Vermelho
B	Azul
C	Ciano
M	Magenta
Y	Amarelo
K	Preto
W	Branco

	Tipo de marcador
+	Cruz
o	Circunferência
*	Asterisco
.	Ponto
S	Quadrado
D	Losango
P	Estrela de 5 pontas
H	Estrela de 6 pontas

	Tipos de linha
-	Linha sólida
–	Linha tracejada
-.	Linha traço e ponto
:	Linha pontilhada

- **hold** - mantém a figura atual para que o próximo plot seja feito por cima do anterior.
- **grid** - desenha uma grade pontilhada na figura.
- **xlabel** - define um rótulo para o eixo x do gráfico.
- **ylabel** - define um rótulo para o eixo y do gráfico.
- **title** - define um título para a figura.
- **axis** - define os limites do gráfico.
- **axis equal** - equaliza o tamanho das medidas dos eixos do gráfico.
- **drawnow** - dá prioridade máxima momentânea ao plot. Do contrário, os plots têm prioridade baixa.
- **semilogx** - similar ao plot, mas com escala logarítmica no eixo x.
- **semilogy** - similar ao plot, mas com escala logarítmica no eixo y.
- **loglog** - similar ao plot, mas com escala logarítmica nos dois eixos.
- **subplot** - particiona uma figura em partes iguais para receber diferentes plots.

Abaixo alguns exemplos de plots e seus resultados:

```octave
x = 0:1:2*pi;
y = cos(x);

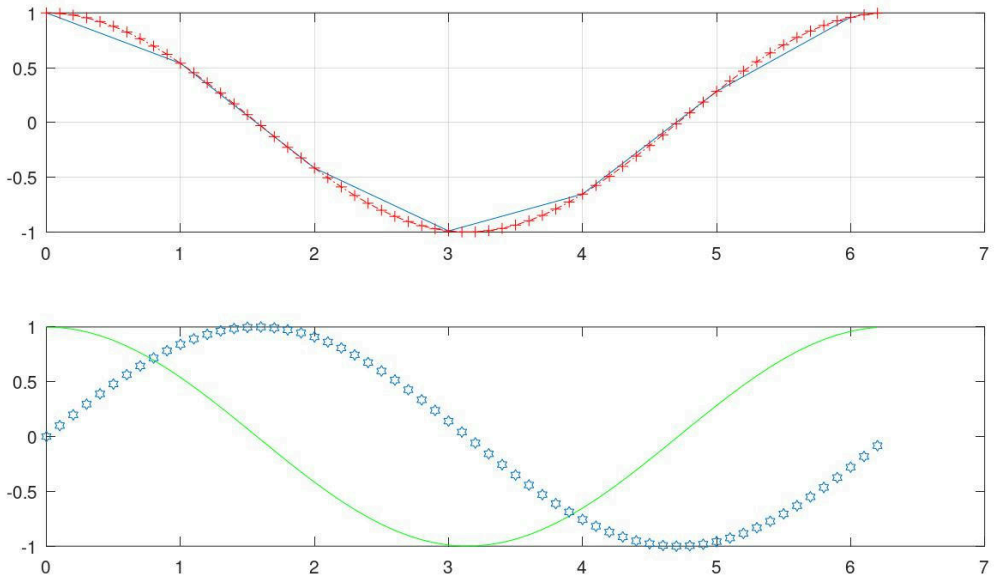
x2 = 0:0.1:2*pi;
y2 = cos(x2);

y3 = sin(x2);

subplot(2,1,1)
plot(x,y)
hold on
grid on
plot(x2,y2,"r+-.")

subplot(2,1,2)
plot(x2,y2,"g")
hold on
plot(x2,y3,"h")
```

O resultado do código acima é:


1
0.5
0
-0.5
-1
0
1
2
3
4
5
6
7

1
0.5
0
-0.5
-1
0
1
2
3
4
5
6
7

Gráficos 3D

- **plot3** - similar ao plot (plota uma curva) mas no 3D.
- Para o plote de funções no 3D (**plot de superfícies**) o Octave tem algumas possibilidades. Destacarei algumas aqui:
  - mesh - plot de superfície "vazada".
  - surf - plot de superfície "preenchida".
- Para plotar uma **superfície z** definida em função de x e y, precisamos de **dados espalhados no plano (x,y).** Esses pontos espalhados, por simplicidade, são representados como **argumentos matriciais**, em oposição aos argumentos vetoriais usados nos plots de curvas.
- **meshgrid** - função que facilita a criação de pontos espalhados no plano (x,y).

```
>> [Mx, My] = meshgrid(0:1:2)
Mx =

   0   1   2
   0   1   2
   0   1   2

My =

   0   0   0
   1   1   1
   2   2   2
```
Octave

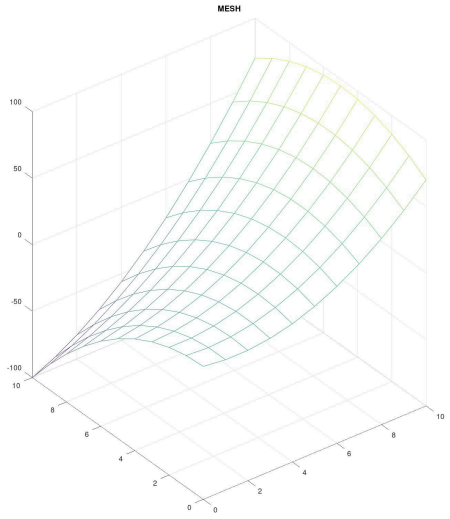
Abaixo um exemplo de utilização de plots de superfície e seu resultado:

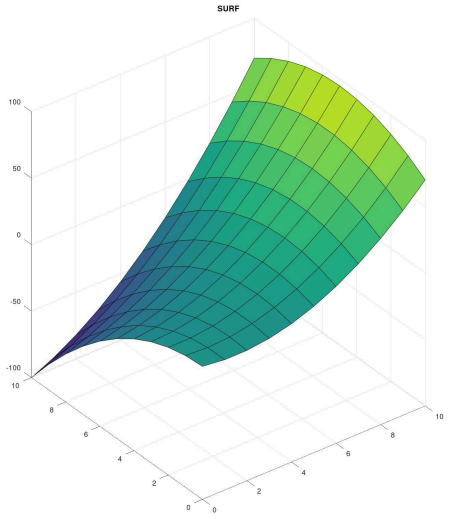
```
[Mx, My] = meshgrid(0:1:10);
Mz = Mx.^2 - My.^2 + Mx.*My - 3*Mx;

subplot(121)
mesh(Mx,My,Mz)
title("MESH")

subplot(122)
surf(Mx,My,Mz)
title("SURF")
```
Octave

O resultado do código acima é:


MESH
100
50
0
-50
-100
10
8
6
4
2
0
0
2
4
6
8
10


SURF
100
50
0
-50
-100
10
8
6
4
2
0
0
2
4
6
8
10